

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
44.03.04 Профессиональное обучение (по отраслям)

направленность (профиль)
«Информатика и искусственный интеллект в образовании»

Курсовая работа
по дисциплине «Прикладные информационные технологии»

«Развёртывание сервиса мониторинга доступности Uptime Kuma в Docker с использованием готового образа, обратного прокси nginx-proxy и автоматическим выпуском сертификатов Let's Encrypt»

Обучающегося 3 курса
очной формы обучения
Павленко Павел Евгеньевич

Руководитель курсовой работы:
старший преподаватель кафедры
ИТЭО
Аксютин Павел Александрович

Санкт-Петербург
2026

СОДЕРЖАНИЕ

<u>1. ОБЗОР И СРАВНЕНИЕ ТЕХНОЛОГИЙ И СЕРВИСОВ.....</u>	<u>5</u>
1.1. ПЛАТФОРМА КОНТЕЙНЕРИЗАЦИИ	5
1.2. СРЕДСТВО ОПИСАНИЯ МНОГОКОНТЕЙНЕРНОЙ КОНФИГУРАЦИИ	6
1.3. РАЗВЁРТЫВАЕМОЕ ПРИЛОЖЕНИЕ	6
1.4. ОБРАТНЫЙ ПРОКСИ-СЕРВЕР	7
1.5. АВТОМАТИЗАЦИЯ ВЫПУСКА TLS-СЕРТИФИКАТОВ	8
1.6. ХРАНЕНИЕ ДАННЫХ ПРИЛОЖЕНИЯ	9
<u>2. СИСТЕМНЫЕ ТРЕБОВАНИЯ РАЗВЁРТЫВАЕМОГО ПРИЛОЖЕНИЯ</u>	<u>10</u>
2.1. СЕРВЕРНАЯ СРЕДА.....	10
2.2. ПРОГРАММНЫЕ ТРЕБОВАНИЯ ПРИЛОЖЕНИЯ.....	10
2.3. ИСПОЛЬЗУЕМЫЕ БАЗОВЫЕ ОБРАЗЫ	11
<u>3. ПРОГРАММНЫЙ КОД И КОНФИГУРАЦИОННЫЕ ФАЙЛЫ.....</u>	<u>12</u>
3.1. ФАЙЛ ПЕРЕМЕННЫХ ОКРУЖЕНИЯ (.ENV)	13
3.2. ОСНОВНОЙ ФАЙЛ КОНФИГУРАЦИИ (DOCKER-COMPOSE.YML)	14
<u>4. ДОКУМЕНТАЦИЯ ПО ЗАПУСКУ ПРОЕКТА</u>	<u>16</u>
4.1. ПРЕДВАРИТЕЛЬНЫЕ ТРЕБОВАНИЯ	16
4.2. УСТАНОВКА DOCKER	16
4.3. ПОДГОТОВКА ФАЙЛОВ ПРОЕКТА.....	17
4.4. ЗАПУСК	17
4.5. ПРОВЕРКА РАБОТОСПОСОБНОСТИ.....	18
4.6. ОСТАНОВКА И ОБНОВЛЕНИЕ	18
<u>5. ХОД РАЗВЁРТЫВАНИЯ И РЕШЁННЫЕ ПРОБЛЕМЫ</u>	<u>19</u>
5.1. ПОДГОТОВКА СЕРВЕРА, УСТАНОВКА DOCKER И НАСТРОЙКА DNS	19
5.2. ПОВРЕЖДЕНИЕ ФАЙЛА DOCKER-COMPOSE.YML ПРИ РЕДАКТИРОВАНИИ.....	20
5.3. ОШИБКА ОБНАРУЖЕНИЯ КОНТЕЙНЕРА ОБРАТНОГО ПРОКСИ.....	20
5.4. УСТАРЕВШИЙ ТЕГ ОБРАЗА ПРИЛОЖЕНИЯ	21
5.5. ВЫПУСК СЕРТИФИКАТА И ПРОВЕРКА РЕЗУЛЬТАТА	21
<u>ЗАКЛЮЧЕНИЕ.....</u>	<u>23</u>
<u>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ</u>	<u>25</u>

ВВЕДЕНИЕ

Современная эксплуатация веб-сервисов и серверной инфраструктуры немыслима без систем непрерывного контроля их доступности. Отказ сайта, API или иного сетевого сервиса, оставшийся незамеченным в течение продолжительного времени, приводит к финансовым и репутационным потерям. Средства мониторинга доступности (uptime-мониторинг) решают эту задачу, регулярно опрашивая контролируемые узлы и оповещая ответственных лиц при возникновении сбоя.

Одновременно с этим неотъемлемым требованием к любому современному веб-сервису стало шифрование канала связи по протоколу HTTPS. Ручное получение и продление TLS-сертификатов является трудоёмкой и подверженной ошибкам задачей, поэтому особую практическую ценность приобретает автоматизация этого процесса.

Целью настоящей курсовой работы является развёртывание сервиса мониторинга доступности Uptime Kuma в контейнеризованной среде Docker с использованием готового образа приложения, собственной конфигурации Docker Compose, обратного прокси-сервера nginx-проху и автоматическим выпуском и продлением сертификатов Let's Encrypt.

Для достижения поставленной цели решаются следующие задачи:

1. провести обзор и сравнение технологий и сервисов, применяемых в работе, и обосновать их выбор;
2. определить системные требования развёртываемого приложения и состав используемых базовых образов;
3. разработать конфигурационные файлы инфраструктуры: файл переменных окружения и файл Docker Compose;
4. развернуть приложение на сервере с настройкой обратного прокси и автоматическим выпуском сертификата Let's Encrypt;
5. составить краткую документацию по запуску проекта.

Объектом исследования выступает процесс контейнеризации и развёртывания веб-приложения с обеспечением защищённого доступа, предметом — конкретная реализация данного процесса средствами Docker, Docker Compose, nginx-проxy и asme-companion на примере сервиса Uptime Kuma.

1. Обзор и сравнение технологий и сервисов

В данном разделе рассматриваются технологии и сервисы, задействованные в работе. Для каждой группы приводятся альтернативные решения, которые могли бы быть использованы вместо выбранных, и обосновывается сделанный выбор.

1.1. Платформа контейнеризации

В качестве платформы контейнеризации выбран Docker — наиболее распространённое средство создания и запуска контейнеров, обладающее обширной документацией и экосистемой готовых образов. Альтернативами являются Podman — демон-независимая среда, совместимая с командами Docker, а также системы уровня оркестрации, например Kubernetes.

Docker выбран по следующим причинам: низкий порог вхождения, зрелая документация, широкая распространённость в учебных и производственных сценариях, а также штатная поддержка Docker Compose — инструмента, необходимого для описания многоконтейнерной конфигурации данного проекта. Kubernetes, являясь полноценной системой оркестрации, избыточен для задачи развёртывания одного небольшого сервиса на единственном сервере и существенно усложнил бы конфигурацию без практической пользы в рамках учебной работы. Сравнение рассматриваемых средств контейнеризации представлено в таблице 1.

Таблица 1 — Сравнение средств контейнеризации

Критерий	Docker	Podman	Kubernetes
Порог входа	Низкий	Средний	Высокий
Поддержка Docker Compose	Штатная	Частичная (podman-compose)	Не применяется
Оркестрация нескольких узлов	Отсутствует	Отсутствует	Полноценная
Применимость к задаче	Оптимально	Приемлемо	Избыточно

1.2. Средство описания многоконтейнерной конфигурации

Для описания и совместного запуска связанных контейнеров используется Docker Compose. Альтернативой могло бы стать последовательное управление каждым контейнером через отдельные команды `docker run` либо применение оркестратора. Docker Compose позволяет декларативно описать все сервисы, сети и тома проекта в одном YAML-файле, что обеспечивает воспроизводимость развёртывания одной командой и наглядность архитектуры — это существенно как для эксплуатации, так и для защиты работы.

1.3. Развёртываемое приложение

В качестве развёртываемого приложения выбран Uptime Kuma — свободный самостоятельно размещаемый (self-hosted) сервис мониторинга доступности сайтов и сетевых сервисов с открытым исходным кодом. Среди аналогов можно назвать Gatus, Statping-ng и коммерческий облачный сервис UptimeRobot.

Uptime Kuma выбран по совокупности следующих причин: наличие образа, публикуемого и поддерживаемого разработчиками проекта, в публичном реестре Docker Hub, что позволяет использовать готовый,

поддерживаемый разработчиком образ без самостоятельной сборки; встроенное хранение данных в SQLite внутри тома, не требующее развёртывания отдельной системы управления базами данных; наглядный веб-интерфейс с графиками времени отклика, удобный для демонстрации результатов работы; умеренные требования к ресурсам, что важно при развёртывании на бюджетном виртуальном сервере. Сравнение сервисов мониторинга доступности приведено в таблице 2.

Таблица 2 — Сравнение сервисов мониторинга доступности

Критерий	Uptime Kuma	Gatus	UptimeRobot
Модель распространения	Self-hosted, open-source	Self-hosted, open-source	Облачный сервис (SaaS)
Официальный образ Docker	Есть	Есть	Не применимо
Хранение данных	Встроенный SQLite	Конфигурация в YAML	На стороне провайдера
Веб-интерфейс	Полноценный, с графиками	Ограниченный	Полноценный
Контроль над данными	Полный (свой сервер)	Полный (свой сервер)	Отсутствует

Gatus привлекателен минимальными требованиями к ресурсам, однако его веб-интерфейс существенно уступает по функциональности, а конфигурация полностью статична и задаётся файлом, что менее удобно для демонстрации динамического добавления мониторов. UptimeRobot исключён из рассмотрения, поскольку является облачным сервисом и не соответствует теме курсовой работы, посвящённой самостоятельному развёртыванию и контейнеризации.

1.4. Обратный прокси-сервер

Для маршрутизации входящих запросов и терминирования HTTPS используется обратный прокси на базе образа nginx-проху. Альтернативами

являются Traefik, Caddy и ручная настройка типового NGINX с самостоятельно прописанными конфигурационными файлами для каждого домена.

Решение nginx-proxy выбрано потому, что оно автоматически обнаруживает запущенные контейнеры приложений по их переменным окружения и формирует конфигурацию NGINX без ручного вмешательства, а также имеет штатный сопутствующий контейнер для интеграции с Let's Encrypt. Это позволяет добавлять новые приложения на сервер без редактирования конфигурации самого прокси — достаточно запустить контейнер с нужными переменными окружения. Сравнение решений обратного прокси приведено в таблице 3.

Таблица 3 — Сравнение решений обратного прокси

Критерий	nginx-proxy	Traefik / Caddy	Ручной NGINX
Автообнаружение контейнеров	По переменным окружения	По меткам / Caddyfile	Отсутствует
Автоматический TLS	Через acme-companion	Встроенный	Требуется Certbot вручную
Порог входа для новичка	Низкий	Средний	Высокий

1.5. Автоматизация выпуска TLS-сертификатов

Автоматический выпуск и продление сертификатов обеспечивает acme-companion — вспомогательный контейнер, работающий в связке с nginx-proxy. Он отслеживает контейнеры с переменной окружения LETSENCRYPT_HOST, обращается в удостоверяющий центр Let's Encrypt по протоколу ACME, проходит проверку владения доменом, получает сертификат и в дальнейшем автоматически продлевает его до истечения девяностодневного срока действия.

Альтернативой является самостоятельный запуск клиента Certbot, однако он требует отдельной настройки расписания продления через cron и

интеграции с веб-сервером, тогда как асме-companion выполняет обе задачи автоматически, что и определило его выбор для данного проекта.

1.6. Хранение данных приложения

В отличие от многих self-hosted приложений, требующих отдельной системы управления базой данных, Uptime Kuma использует встроенную базу SQLite, файл которой хранится в именованном томе Docker. Такое решение соответствует выбранному в рамках задания варианту развёртывания — использованию готового образа приложения с собственной конфигурацией Docker Compose, без разработки многоконтейнерной архитектуры «приложение — база данных». Данные (список мониторов, история проверок, настройки уведомлений) сохраняются в томе и не теряются при обновлении или пересоздании контейнера приложения.

2. Системные требования развёртываемого приложения

В данном разделе описываются программная среда, зависимости и базовые образы, необходимые для работы приложения Uptime Kuma в контейнеризованном виде.

2.1. Серверная среда

Развёртывание выполняется на виртуальном сервере (VDS) провайдера Selectel со следующими характеристиками: операционная система Ubuntu 24.04 LTS, публичный IPv4-адрес, открытые сетевые порты 80 и 443. В данной работе наличие публичного адреса и настроенной A-записи домена является необходимым условием прохождения проверки владения доменом (HTTP-01 challenge) при выпуске сертификата в реализованной конфигурации.

На сервер устанавливается среда выполнения контейнеров Docker Engine и официальный плагин Docker Compose. Межсетевой экран ufw на сервере не был включён и настроен, а дополнительная фильтрация трафика на уровне облачной инфраструктуры провайдером не применялась, что не создаёт дополнительных препятствий для прохождения трафика к портам 80 и 443.

2.2. Программные требования приложения

Uptime Kuma реализован на платформе Node.js с использованием фреймворка Vue.js для клиентской части интерфейса. Все необходимые зависимости включены непосредственно в официальный образ контейнера, что избавляет от необходимости их самостоятельной установки. Внутри контейнера приложение слушает порт 3001, а инициализация процесса выполняется через dumb-init — минималистичную обёртку, корректно

обрабатывающую системные сигналы внутри контейнера. Используемые программные компоненты приложения перечислены в таблице 4.

Таблица 4 — Программные компоненты приложения

Компонент	Назначение
Node.js	Среда выполнения серверной части приложения
Vue.js	Фреймворк клиентского веб-интерфейса
SQLite	Встроенное хранилище мониторов и истории проверок
dumb-init	Корректная обработка сигналов процесса внутри контейнера

2.3. Используемые базовые образы

Проект использует три образа, публикуемые и поддерживаемые разработчиками соответствующих проектов, из публичного реестра Docker Hub:

- `louislam/uptime-kuma:2` — образ приложения мониторинга (<https://hub.docker.com/r/louislam/uptime-kuma>);
- `nginxproxy/nginx-proxy:latest` — образ обратного прокси-сервера (<https://hub.docker.com/r/nginxproxy/nginx-proxy>);
- `nginxproxy/acme-companion:latest` — образ компаньона для автоматического выпуска сертификатов Let's Encrypt (<https://hub.docker.com/r/nginxproxy/acme-companion>).

Для приложения намеренно указан тег версии 2, а не тег `latest`: в данном образе тег `latest` исторически закреплён за устаревшей и более не поддерживаемой первой версией приложения (подробнее об этом — в разделе 5.5).

3. Программный код и конфигурационные файлы

Инфраструктура проекта описывается двумя файлами: файлом переменных окружения (.env) и основным файлом Docker Compose (docker-compose.yml). Оба файла размещаются в одном каталоге проекта на сервере и включены в состав репозитория, за исключением .env, который по соображениям безопасности не публикуется (вместо него в репозиторий включён шаблон .env.example).

На рисунке 1 представлена архитектура развёрнутой системы. Входящие запросы пользователя по протоколу HTTPS поступают на обратный прокси nginx-проху, который определяет целевой контейнер по заданному доменному имени и перенаправляет запросы на Uptime Kuma по внутреннему порту 3001. Контейнеры nginx-проху, acme-companion и Uptime Kuma объединены общей Docker-сетью проху-net. Контейнер acme-companion взаимодействует с удостоверяющим центром Let's Encrypt по протоколу ACME, выполняет проверку владения доменом и получает TLS-сертификат, после чего сертификат используется обратным прокси для организации защищённого HTTPS-соединения.



Рисунок 1 — Архитектура развёрнутой системы мониторинга Uptime Kuma

3.1. Файл переменных окружения (.env)

Изменяемые параметры — доменное имя и контактный адрес электронной почты для уведомлений Let's Encrypt — вынесены в отдельный файл .env. Такое разделение отделяет конфигурацию окружения от описания инфраструктуры и позволяет развернуть проект на другом домене без изменения основного файла Compose.

```
DOMAIN=appacher.ru
LETSencrypt_EMAIL=pe.pavlenko@mail.ru
```

3.2. Основной файл конфигурации (docker-compose.yml)

Файл описывает три сервиса (nginx-proxy, acme-companion, uptime-kuma), объединённые общей изолированной сетью proxy-net, а также именованные тома для хранения конфигурации прокси, сертификатов и данных приложения. Контейнер приложения не публикует порты наружу напрямую — весь трафик проходит только через обратный прокси, что позволяет в дальнейшем разместить на том же сервере произвольное количество других приложений на портах 80 и 443 без конфликтов.

Для того чтобы компаньон acme-companion корректно обнаруживал контейнер обратного прокси, последнему присвоена метка com.github.nginx-proxy.nginx=true. Связь приложения с прокси и системой выпуска сертификатов обеспечивается четырьмя переменными окружения: VIRTUAL_HOST указывает прокси, на какой домен направлять трафик контейнера, VIRTUAL_PORT — на каком порту приложение принимает соединения внутри контейнера, а LETSENCRYPT_HOST и LETSENCRYPT_EMAIL сообщают компаньону, для какого домена и на какой адрес выпускать сертификат.

```
services:
  nginx-proxy:
    image: nginxproxy/nginx-proxy:latest
    container_name: nginx-proxy
    restart: unless-stopped
    labels:
      - com.github.nginx-proxy.nginx=true
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - conf:/etc/nginx/conf.d
      - vhost:/etc/nginx/vhost.d
      - html:/usr/share/nginx/html
      - certs:/etc/nginx/certs:ro
      - /var/run/docker.sock:/tmp/docker.sock:ro
    networks:
      - proxy-net

  acme-companion:
    image: nginxproxy/acme-companion:latest
    container_name: acme-companion
```

```

restart: unless-stopped
depends_on:
  - nginx-proxy
environment:
  - DEFAULT_EMAIL=${LETSENCRYPT_EMAIL}
volumes:
  - conf:/etc/nginx/conf.d
  - vhost:/etc/nginx/vhost.d
  - html:/usr/share/nginx/html
  - certs:/etc/nginx/certs
  - acme:/etc/acme.sh
  - /var/run/docker.sock:/var/run/docker.sock:ro
networks:
  - proxy-net

uptime-kuma:
  image: louislam/uptime-kuma:2
  container_name: uptime-kuma
  restart: unless-stopped
  volumes:
    - uptime-kuma-data:/app/data
  environment:
    - VIRTUAL_HOST=${DOMAIN}
    - VIRTUAL_PORT=3001
    - LETSENCRYPT_HOST=${DOMAIN}
    - LETSENCRYPT_EMAIL=${LETSENCRYPT_EMAIL}
  networks:
    - proxy-net

networks:
  proxy-net:
    name: proxy-net

volumes:
  conf:
  vhost:
  html:
  certs:
  acme:
  uptime-kuma-data:

```

4. Документация по запуску проекта

В данном разделе приведена краткая инструкция по развёртыванию проекта, оформленная по образцу описаний, публикуемых на Docker Hub, и размещённая также в файле README.md репозитория проекта. Инструкция предполагает наличие сервера с публичным IPv4-адресом и зарегистрированного доменного имени, А-запись которого указывает на адрес сервера.

4.1. Предварительные требования

- сервер под управлением ОС Linux с публичным IPv4-адресом;
- зарегистрированное доменное имя с А-записью, указывающей на адрес сервера;
- свободные порты 80 и 443 на сервере.

4.2. Установка Docker

Установка Docker Engine и плагина Compose выполняется из официального репозитория Docker (на примере Ubuntu):

```
apt update && apt upgrade -y
apt install -y ca-certificates curl gnupg git
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
"$VERSION_CODENAME") stable" | \
  tee /etc/apt/sources.list.d/docker.list > /dev/null
apt update
apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

Корректность установки проверяется запуском тестового образа:

```
docker run hello-world
```

4.3. Подготовка файлов проекта

Клонирование репозитория проекта и подготовка файла переменных окружения на основе шаблона:

```
git clone https://github.com/aPPacheR/appacher-monitor.git
cd appacher-monitor
cp .env.example .env
nano .env # указать свой домен и email
```

4.4. Запуск

```
docker compose up -d
```

Команда скачивает образы, создаёт изолированную сеть проху-net, именованные тома и запускает все три контейнера в фоновом режиме. Проверка статуса контейнеров выполняется командой:

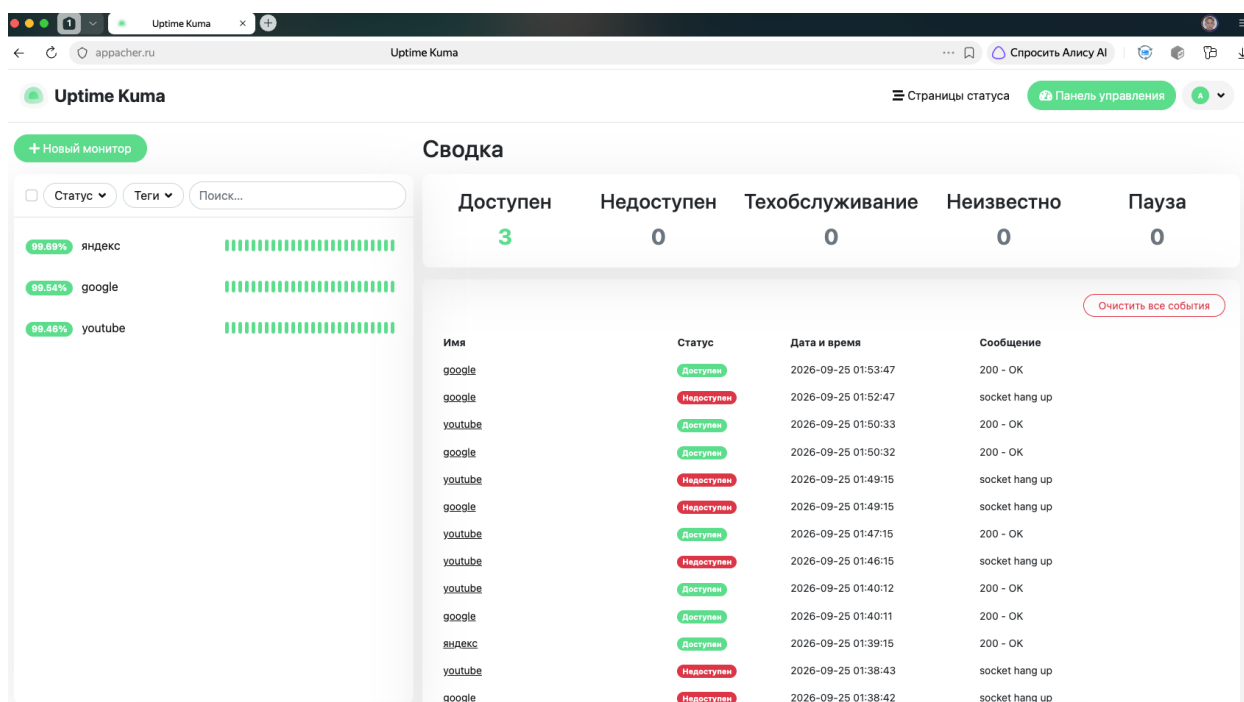
```
docker compose ps
```

```
root@streaming-meije:~/appacher-monitor# docker compose up -d
[+] up 3/3
✓ Container uptime-kuma    Running      0.0s
✓ Container nginx-proxy    Started     1.4s
✓ Container acme-companion Started     0.0s
root@streaming-meije:~/appacher-monitor# docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
acme-companion	nginxproxy/acme-companion:latest	"/bin/bash /app/entr..."	acme-companion	6 minutes ago	Up 28 second
nginx-proxy	nginxproxy/nginx-proxy:latest	"/app/docker-entrypo..."	nginx-proxy	44 seconds ago	Up 43 second
uptime-kuma	louislam/uptime-kuma:latest	"/usr/bin/dumb-init ..."	uptime-kuma	6 minutes ago	Up 6 minutes
(healthy)	3001/tcp				

4.5. Проверка работоспособности

После запуска и распространения DNS-записи приложение доступно по адресу `http://<домен>` — на этом этапе сертификат ещё не выпущен, так как компаньону `асте-компаньон` требуется убедиться, что домен указывает на данный сервер. После успешного прохождения проверки владения доменом приложение становится доступно по защищённому адресу `https://<домен>` с действующим сертификатом Let's Encrypt.



4.6. Остановка и обновление

Остановка всех контейнеров с сохранением томов данных:

```
docker compose down
```

Обновление образов до новых версий с последующим пересозданием контейнеров:

```
docker compose pull
docker compose up -d
```

5. Ход развёртывания и решённые проблемы

В данном разделе описывается практический ход развёртывания проекта на сервере, включая наиболее показательные трудности, возникшие в процессе, и способы их решения. Развёртывание выполнялось на виртуальном сервере провайдера Selectel с доменным именем appacher.ru, зарегистрированным у регистратора reg.ru.

5.1. Подготовка сервера, установка Docker и настройка DNS

После аренды сервера и настройки доступа по протоколу SSH была установлена среда Docker Engine и плагин Docker Compose по методике, приведённой в разделе 4.2. Корректность установки подтверждена успешным запуском тестового образа hello-world.

Домен appacher.ru был зарегистрирован у одного регистратора (reg.ru), тогда как управление DNS-зоной домена было решено делегировать серверам провайдера хостинга Selectel. Для этого в DNS-зоне на стороне Selectel добавлена ресурсная запись типа A, связывающая домен с публичным IP-адресом сервера, а в панели управления регистратора reg.ru в качестве DNS-серверов домена явно указаны серверы имён Selectel. Без выполнения второго шага запись A оставалась бы неактивной, поскольку регистратор по умолчанию продолжал бы использовать собственные DNS-серверы, не содержащие нужной записи.

Распространение изменений в системе доменных имён заняло около двух часов, после чего домен начал корректно разрешаться в IP-адрес сервера, что было подтверждено командой `dig appacher.ru +short`.

```
lappacher@aPPacheRs-MacBook-Air ~ % dig appacher.ru +short
5.53.124.183
```

5.2. Повреждение файла `docker-compose.yml` при редактировании

При первой попытке создания файла `docker-compose.yml` через текстовый редактор `nano` с последующей вставкой заранее подготовленного текста через буфер обмена по SSH-соединению возникла ошибка синтаксического анализатора формата YAML: «mapping values are not allowed in this context». Причиной оказалась функция автоматического повторения отступов, встроенная в `nano`: при построчной вставке многострочного текста редактор добавлял к каждой новой строке отступ предыдущей, из-за чего критичная для формата YAML структура отступов была нарушена.

Проблема была устранена переходом от интерактивного редактирования к созданию файла единой командой с использованием конструкции `heredoc` в оболочке `bash` (`cat > docker-compose.yml << 'EOF' ... EOF`), которая записывает переданный текст в файл целиком, без построчной обработки редактором. Аналогичный приём применялся и при последующих правках файла.

```
cat > docker-compose.yml << 'EOF'
services:
    ...
EOF
```

5.3. Ошибка обнаружения контейнера обратного прокси

После первого успешного запуска стека контейнер `asme-companion` находился в статусе `Restarting` и циклически перезапускался. В журнале контейнера содержалась ошибка: «Error: can't get nginx-proxy container ID!». Причина заключалась в том, что используемая версия компаньона более не определяет контейнер обратного прокси автоматически по имени образа и требует явного указания — либо через переменную окружения `NGINX_PROXY_CONTAINER`, либо через метку контейнера.

Проблема была устранена добавлением в описание сервиса `nginx-proxy` метки `com.github.nginx-proxy.nginx=true` (см. листинг в разделе 3.2) с последующим пересозданием контейнера командой `docker compose up -d`.

После этого `acme-companion` корректно обнаружил контейнер прокси и перешёл в устойчивый режим работы.

```
docker compose logs acme-companion
# Error: can't get nginx-proxy container ID !
```

5.4. Устаревший тег образа приложения

После первого успешного открытия веб-интерфейса приложение вывело предупреждение об использовании устаревшей и более не поддерживаемой версии (1.23.17), несмотря на то что в конфигурации был указан тег `latest`. Проверка документации образа на Docker Hub показала, что тег `latest` для `louislam/uptime-kuma` по историческим причинам сохранения обратной совместимости продолжает указывать на последний релиз первой версии приложения, тогда как актуальной и рекомендованной к использованию является вторая версия, распространяемая под тегом 2.

Проблема была устранена изменением тега образа в `docker-compose.yml` с `louislam/uptime-kuma:latest` на `louislam/uptime-kuma:2` с последующим обновлением образа и пересозданием контейнера:

```
docker compose pull uptime-kuma
docker compose up -d
```

Этот случай наглядно демонстрирует, что тег `latest` не всегда соответствует самой новой версии программного обеспечения и при развёртывании `production`-сервисов требует проверки по официальной документации образа.

5.5. Выпуск сертификата и проверка результата

После устранения перечисленных проблем компаньон `acme-companion` автоматически обратился в удостоверяющий центр Let's Encrypt, прошёл проверку владения доменом по протоколу HTTP-01 и получил сертификат,

Приложение стало доступно по адресу <https://appacher.ru> с действующим защищённым соединением. Был создан первый административный аккаунт, после чего в интерфейсе настроено несколько тестовых мониторов типа HTTP(s), подтвердивших работоспособность системы мониторинга в целом.



ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была достигнута поставленная цель — выполнено развёртывание сервиса мониторинга доступности Uptime Kuma в контейнеризованной среде Docker с организацией защищённого доступа по протоколу HTTPS. Использован готовый образ приложения, поддерживаемый разработчиком проекта, для которого разработана собственная конфигурация Docker Compose, включающая обратный прокси-сервер и автоматический выпуск сертификатов Let's Encrypt.

В рамках работы были решены все поставленные задачи: проведён обзор и сравнение применяемых технологий с обоснованием выбора; определены системные требования и состав базовых образов; разработаны конфигурационные файлы инфраструктуры; выполнено развёртывание с настройкой обратного прокси и автоматическим выпуском сертификата Let's Encrypt; составлена документация по запуску проекта, размещённая в общедоступном репозитории.

Практическая ценность работы заключается в получении воспроизводимой конфигурации, которая может быть развёрнута на произвольном сервере одной командой и обеспечивает автоматическое поддержание TLS-сертификата в актуальном состоянии без участия администратора. Следует отметить, что образы nginx-proxu и acme-companion в конфигурации закреплены плавающим тегом latest, что для учебной работы допустимо, но при промышленной эксплуатации предполагает либо фиксацию конкретных протестированных версий образов, либо контроль изменений окружения при повторных развёртываниях. В процессе развёртывания был приобретён практический опыт диагностики и устранения типичных проблем контейнеризованных приложений: повреждения конфигурационных файлов при интерактивном редактировании, взаимного обнаружения связанных контейнеров и корректного выбора версии образа при развёртывании. Полученные навыки контейнеризации, проектирования многоконтейнерных

конфигураций и автоматизации инфраструктурных задач имеют непосредственное прикладное значение.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Uptime Kuma : a fancy self-hosted monitoring tool : [сайт]. — GitHub. — URL: <https://github.com/louislam/uptime-kuma> (дата обращения: 25.09.2026). — Текст : электронный.
2. Uptime Kuma : образ контейнера : [сайт]. — Docker Hub. — URL: <https://hub.docker.com/r/louislam/uptime-kuma> (дата обращения: 25.09.2026). — Текст : электронный.
3. Docker documentation : [сайт]. — URL: <https://docs.docker.com/> (дата обращения: 25.09.2026). — Текст : электронный.
4. Docker Compose overview : [сайт]. — URL: <https://docs.docker.com/compose/> (дата обращения: 25.09.2026). — Текст : электронный.
5. nginx-proxy : automated nginx proxy for Docker containers : [сайт]. — GitHub. — URL: <https://github.com/nginx-proxy/nginx-proxy> (дата обращения: 25.09.2026). — Текст : электронный.
6. acme-companion : Let's Encrypt companion container for nginx-proxy : [сайт]. — GitHub. — URL: <https://github.com/nginx-proxy/acme-companion> (дата обращения: 25.09.2026). — Текст : электронный.
7. Let's Encrypt : a nonprofit Certificate Authority : [сайт]. — URL: <https://letsencrypt.org/> (дата обращения: 25.09.2026). — Текст : электронный.
8. Automatic Certificate Management Environment (ACME) : RFC 8555 : [сайт]. — Internet Engineering Task Force. — URL: <https://www.rfc-editor.org/rfc/rfc8555> (дата обращения: 25.09.2026). — Текст : электронный.
9. Get Docker : [сайт]. — Docker documentation. — URL: <https://docs.docker.com/get-started/get-docker/> (дата обращения: 25.09.2026). — Текст : электронный.